

Interview prep · Agentic RAG project

50 questions they'll ask about your PDF QA Bot

Every answer uses the real numbers from the codebase: 1,200-character chunks, 768-dimension embeddings, a 0.35 similarity threshold and a 5-step agent limit. Read each answer once, then say it in your own words without looking.

- Pitch & product Q1–Q5
- RAG fundamentals Q6–Q12
- Embeddings & vector search Q13–Q19
- Ingestion & chunking Q20–Q25
- Agent & tool calling Q26–Q32
- Security & multi-tenancy Q33–Q39
- Reliability & errors Q40–Q45
- Testing, scale & trade-offs Q46–Q50

The architecture you'll be asked to draw

Upload (owner, signed in)

PDF → unpdf, per page → Chunk 1200 / 200 overlap → Embed 768-d, normalised → pgvector + HNSW

Chat (customer, no login)

Question → Agent loop, max 5 steps → searchDocs or createTicket → match_chunks(owner_id) → Answer + page citations

Pitch & product

Q1–Q5

Q1 Explain the project in 60 seconds.

Small businesses answer the same customer questions all day, even though the answers already sit in a PDF. PDF QA Bot lets an owner upload that PDF and get a public chat link. Customers ask in plain language, and an AI agent searches the document, answers only from it with the page number, and raises a ticket for the owner when the document has no answer. It's built with Next.js 15, Supabase (Postgres, pgvector, Auth, Storage) and the Gemini API, deployed on Vercel.

Q2 Why build this instead of a generic "chat with PDF" app?

A generic app answers the person who uploaded the file. Mine separates two users: the owner who uploads, and many anonymous customers who ask. That forced real engineering problems: multi-tenant isolation, a public endpoint with no login, honest refusal, and a ticket flow that turns unanswered questions into a to-do list for the owner.

Q3 Who are the users and what does each one see?

The owner signs in with email and password, uploads PDFs, copies the chat link and reads a ticket inbox. The customer opens `/chat/<businessId>` with no account, asks questions and sees answers with tool badges (`searchDocs()`) and expandable page citations. Signed-out visitors to `/` see a landing page; signed-in owners see their dashboard at the same URL.

Q4 What feature are you most proud of?

Honest "I don't know". When no passage clears the 0.35 similarity threshold, the search tool returns `found: 0` plus an explicit note telling the model to say it doesn't have the information and offer a ticket. An explicit signal is far more reliable than an empty array the model has to interpret, and in support a confident wrong answer (a wrong price, a wrong refund rule) is worse than no answer.

Q5 What was the hardest bug you hit?

Two stand out. First, `match_chunks` failed with "operator does not exist: vector <=> vector" because I had pinned `search_path = ''`, which hid the pgvector operator living in the `extensions` schema; I fixed it with `set search_path = public, extensions`. Second, login worked in the browser but the server said "fetch failed": Indian ISPs were blocking `supabase.co` with DNS and then SNI blocking. Chrome got through using secure DNS and Encrypted Client Hello, but Node.js didn't. I diagnosed it by logging `err.cause.code` (ECONNRESET) and confirmed it by switching networks.

RAG fundamentals

Q6–Q12

Q6 What is RAG and why use it here?

Retrieval-Augmented Generation means fetching relevant text from your own data and giving it to the LLM as context before it answers. The model doesn't know FitZone's prices, and fine-tuning would be slow, costly and impossible to update per business. With RAG, uploading a new PDF updates the bot instantly, and every answer can point to its source page.

Q7 What makes your RAG "agentic"?

Plain RAG runs a fixed pipeline on every message: embed the question, fetch the top chunks, stuff them into the prompt. Here retrieval is a tool the model chooses to call. It decides whether to search, rewrites the question into a better search phrase, can search again, and can choose a different tool (`createTicket`) when search finds nothing. So "hi" doesn't trigger a pointless search, and "how much is it?" becomes "monthly membership price".

Q8 Walk me through what happens when a customer asks a question.

1. The browser POSTs `{businessId, message, history}` to `/api/chat`.
2. Zod validates the body; the server checks the business has at least one ready document.
3. The agent loop sends the conversation plus tool declarations to Gemini.
4. Gemini replies with a function call, `searchDocs({query})`.
5. The server embeds the query, calls `match_chunks` filtered by owner, and returns passages with page and similarity.
6. Gemini reads them and writes the final answer; the server returns answer, de-duplicated sources and the tool trace.

Q9 Why send conversation history, and why only text?

History lets follow-ups work: "how much is it?" means nothing without the previous turn. I send only role and text, not sources or tool traces, because the model doesn't need them and they cost tokens. History is capped at 20 turns of up to 4,000 characters each so context can't grow without limit.

Q10 How do you stop the model from hallucinating?

Five layers: the system prompt says to answer only from `searchDocs` results and never guess hours, prices or policies; it must search before answering any business question; a 0.35 similarity threshold drops weak matches; an empty result returns an explicit "tell the customer you don't have that" note; and temperature is 0.2 for faithful rather than creative output. The evals then check refusals on questions the PDF doesn't cover (parking, pool, student discount).

Q11 Why temperature 0.2?

Temperature controls randomness in token sampling. For support answers I want the model to repeat what the document says, not paraphrase creatively or invent details. Low temperature makes output more consistent; not zero, so phrasing can still sound natural.

Q12 Why show citations to the customer?

Citations make the answer checkable: the customer can open the source and read the exact passage. They also expose bad retrieval during development, because a wrong page number shows immediately that search picked the wrong chunk.

Embeddings & vector search

Q13–Q19

Q13 What is an embedding?

An embedding is a list of numbers (a vector) that represents the meaning of a piece of text. Texts with similar meaning get vectors that point in similar directions, even with no words in common. That's why "I want to stop being a member" finds the cancellation policy.

Q14 Which embedding model and how many dimensions?

`gemini-embedding-001` at 768 dimensions. Its default is 3072, but 768 is Google's recommended size for retrieval, makes each row about 4x smaller, and fits under pgvector HNSW's 2,000-dimension index limit. The number must match `vector(768)` in the schema, or inserts fail.

Q15 Why do you normalise the vectors?

This model uses Matryoshka representation learning: asking for 768 dimensions returns a prefix of the full 3072-dimension vector, and that prefix is no longer unit length. Cosine similarity assumes unit vectors, so I divide each value by the vector's magnitude before storing. Skipping it quietly hurts recall without any error.

Q16 What are task types, and why different ones for documents and queries?

Chunks are embedded with `RETRIEVAL_DOCUMENT` and questions with `RETRIEVAL_QUERY`. A short question and the passage that answers it are different kinds of text, and telling the model which is which places them closer together in vector space. It improves retrieval at zero extra cost.

Q17 What is cosine similarity and what does <=> do?

Cosine similarity measures the angle between two vectors: 1 means same direction, 0 means unrelated. In pgvector, `<=>` returns cosine distance, so `match_chunks` computes similarity as `1 - (embedding <=> query_embedding)`, filters by the threshold, orders by distance and limits to 5.

Q18 What is an HNSW index and why use it?

Hierarchical Navigable Small World is an approximate nearest-neighbour index: a layered graph where search hops from coarse layers to fine ones instead of comparing the query with every row. It turns a full scan into roughly logarithmic search. The trade-off is that it is approximate (it can occasionally miss the exact best match) and uses more memory and build time than IVFFlat. I created it with `vector_cosine_ops` so it matches the `<=>` operator.

Q19 How did you choose a 0.35 threshold and top-5?

0.35 is a starting point tuned against the sample PDF: real matches scored well above it, and unrelated questions (pool, parking) fell below. Top-5 gives the model enough context for questions spanning two sections without flooding it. Both are constants in `tools.ts`; the eval suite exists so I can change them and immediately see whether answers or refusals got worse.

Ingestion & chunking

Q20–Q25

Q20 Walk me through the upload pipeline.

1. Check the user is signed in, the file is a PDF and it is under 10 MB.
2. Insert a `documents` row with status `processing`, so the UI can show progress.
3. Save the original PDF to Supabase Storage at `<userId>/<docId>.pdf`.
4. Extract text per page with `unpdf` (`mergePages: false` keeps page boundaries).
5. Chunk each page, embed the chunks in batches with `RETRIEVAL_DOCUMENT`.
6. Insert chunks in batches of 100, then mark the document `ready` with its chunk count.

Q21 Why chunk at all? Why not embed the whole PDF?

One vector for a whole document averages every topic into mush, so it matches nothing well. Small chunks each represent one topic, so a question about refunds lands on the refund passage. Chunks are also what get pasted into the prompt, and sending only the 5 relevant ones keeps the context small, cheap and focused.

Q22 Why 1,200 characters with 200 overlap?

Chunk size is the main quality dial. Too small and a chunk loses the context that made it meaningful; too large and it mixes topics, which blurs similarity. About 1,200 characters fits FAQ, policy and menu documents well. The 200-character overlap means an answer that straddles a boundary still appears whole in at least one chunk.

Q23 How does your chunker decide where to split?

Structure first, characters last. It splits on paragraph breaks and packs paragraphs into a chunk until the next one would exceed 1,200 characters. A paragraph longer than a chunk is split by sentence (after `.`, `!` or `?`). Only text with no sentence breaks at all, such as a long table, falls back to a hard character split. Blind every-N-characters splitting cuts sentences in half and embeds badly.

Q24 How do you keep page numbers for citations?

I extract text page by page instead of merging the document, and `chunkPages` chunks each page separately, tagging every chunk with its page number. The page is stored in the `chunks` row and returned by `match_chunks`, so the UI can show "Page 2". The trade-off: a paragraph that runs across a page break becomes two chunks.

Q25 What happens if processing fails halfway?

The whole pipeline is in one try/catch. On any error the document is marked `failed` with the error message, and every chunk already written for it is deleted. A half-indexed document could serve answers from incomplete data, so it's better to fail completely and clearly. Scanned PDFs, which have no text layer, are rejected with a message saying they need OCR, and uploads over 400 chunks are refused to protect the free-tier quota.

Agent & tool calling

Q26–Q32

Q26 How does tool calling (function calling) work?

I send Gemini a list of function declarations: a name, a description and a JSON schema for the parameters. The model never runs code. It replies with a structured request like `searchDocs({"query": "monthly membership price"})`. My server decides whether to honour it, runs the real function, and sends the result back as a `functionResponse` so the model can continue.

Q27 Walk me through your agent loop.

It's a hand-written loop of about 50 lines in `agent.ts`. Each step calls the model with the conversation and tools. If the response has no function calls, that text is the final answer and the loop returns. If it has calls, I append the model's turn, run every requested tool in parallel with `Promise.all`, append the results as a new turn, and loop. It stops after `MAX_STEPS = 5` no matter what.

Q28 Why a hard limit of 5 steps?

Without a ceiling, a model that keeps calling `searchDocs` with slightly different queries loops until it hits the rate limit or the request times out. Five is enough for search, maybe a second search, then answer or ticket. If it runs out, it returns an honest "Sorry, I couldn't work that out" instead of a half-finished answer.

Q29 Why not use LangChain or another agent framework?

The loop is about 50 lines, and I can explain every line. A framework adds abstraction, dependencies and its own bugs for something this small, and it hides the parts that matter in an interview: how tool results are fed back, where the step limit lives and how errors reach the model. I'd consider a framework when I need features like durable multi-step workflows or many agents.

Q30 Why echo the model's own turn back verbatim?

The function-call and function-result turns must stay linked, so the model sees what it asked for next to what came back. Newer Gemini models also attach thought signatures to function calls that must be returned unchanged in the next request. Re-sending the model's original `parts` handles both. I also echo each call's `id` in its response so results match their calls when several run in parallel.

Q31 What happens when a tool fails or the model sends bad arguments?

Errors go back to the model as data instead of crashing the request. A Zod validation failure returns `{error: "Invalid arguments", details: [...]}`, which the model can often fix on its next step (for example a query that was too short). An unknown tool name or an unexpected error returns "Tool failed. Do not retry this call." The UI marks failed calls in red in the tool trace.

Q32 How do you stop the agent creating tickets nobody asked for?

Three places. The `createTicket` description says to call it only when search found nothing and the customer has agreed. The system prompt repeats that it may only offer a ticket and must wait for consent. And the eval suite includes "no ticket without consent" and unknown-question cases with `mustNotCall: ['createTicket']`. During manual testing it correctly refused to create a ticket when "yes, my name is Ankit" didn't follow an unanswered question.

Security & multi-tenancy

Q33–Q39

Q33 How do you make sure one business can never see another's documents?

Two layers that protect different paths. Owner pages use the publishable key with the user's session, so Postgres Row Level Security limits every query to rows where `owner_id = auth.uid()`. The public chat has no signed-in user, so it runs server-side with the secret key, which bypasses RLS; there, isolation comes from always passing `p_owner_id` to `match_chunks` and setting `owner_id` from server context on every insert.

Q34 Why is ownerId not a parameter of the searchDocs tool?

If the model could choose the owner id, a customer could type "search the documents of business 0000…" and the model might comply. That's prompt injection with a database attached. So the model only ever controls `query`. The owner id comes from the URL, is validated as a UUID, and is injected by the server through a `ToolContext` the model never sees. The eval suite tests exactly this attack.

Q35 What is Row Level Security?

RLS is a Postgres feature where policies attached to a table filter which rows each user can read or change, enforced inside the database. Even a query with no `WHERE` clause only returns the caller's rows. That's why the dashboard query `select ... from documents` has no owner filter but still shows only your documents. I deliberately created no insert policy on `chunks` and `tickets`, so browsers can't write to them at all; only the server can.

Q36 Publishable key vs secret key: what's the difference?

The publishable key (`sb_publishable_...`) is safe to ship to the browser because every request made with it is still subject to RLS. The secret key (`sb_secret_...`) bypasses RLS entirely, so it lives only in server code, has no `NEXT_PUBLIC_` prefix (which would inline it into the browser bundle), and is never committed to Git. These replace Supabase's older anon and service_role keys.

Q37 Why is match_chunks SECURITY INVOKER, and why pin its search_path?

`SECURITY INVOKER` means the function runs with the caller's permissions, so it can never silently bypass RLS the way a `SECURITY DEFINER` function could. The `search_path` is pinned to `public, extensions` so a caller can't change which tables or operators the function resolves. Pinning it to an empty string broke the pgvector `⟺` operator, which is the bug from Q5.

Q38 Why validate the model's tool arguments with Zod?

Model output is untrusted input, just like a request body. The model could send an empty query, a 10,000-character question or an unexpected type. Zod enforces limits (query 2–500 characters, ticket question 5–2,000, name up to 100) before anything touches the database, and turns violations into structured errors the model can correct.

Q39 How do you defend against prompt injection inside an uploaded PDF?

A malicious PDF could contain "ignore previous instructions". The system prompt states that passages are business documents, not instructions, and that commands inside them must be treated as text. More importantly, the blast radius is small: the model's only powers are searching its own owner's documents and creating a ticket, so even a successful injection can't read other tenants' data or run anything dangerous.

Reliability & errors

Q40-Q45

Q40 Which errors do you retry, and which do you not?

I retry only transient errors, where the same request can succeed later: 429 (rate limited), 500 (server hiccup), 503 (model overloaded), 504 (deadline exceeded), timeouts and network resets like ECONNRESET. I never retry 400, 401 or 404, because those are my mistakes (bad request, bad key, wrong model name) and repeating them gives the same error. Retrying a 4xx just wastes quota.

Q41 Explain exponential backoff and jitter.

Backoff means waiting longer after each failure (2s, 4s, 8s, 16s) so you don't hammer a server that's already struggling. Jitter adds a random 0–1 second to each wait. Without it, a thousand clients that failed together retry together and knock the server over again, the "thundering herd". If the error says how long to wait (Gemini's `"retryDelay":"41s"`), I use that exact value instead of guessing.

Q42 How do you handle Gemini's free-tier limits?

There are two separate limits: per minute (5 requests on `gemini-3.6-flash`) and per day (20). A per-minute 429 is retried after the server's suggested delay. A per-day 429 fails immediately, because waiting seconds can't help, and any suggested wait over 60 seconds is treated the same. The chat model is configurable through `GEMINI_CHAT_MODEL`, because each model has its own separate quota.

Q43 Why set a timeout on every Gemini request?

Retry only helps when a request fails. A request that hangs never fails, so retry never kicks in and the program waits forever. I saw exactly that: the eval sat silent for 10 minutes on one request. Now every request has a 60-second timeout, and a timeout counts as a transient error, so it's retried.

Q44 How do you turn errors into messages for customers?

Errors have two audiences. The full error goes to the server log with `console.error('[chat]', err)` for the developer. The customer gets a specific, actionable message: "today's usage limit" for a daily quota, "too many questions at once" for 429, "the AI service is busy" for 503 and 504, "couldn't reach its services" for network failures. One generic "Something went wrong" helps neither audience.

Q45 Tell me about a bug caused by how JavaScript loads modules.

The eval script loaded `.env.local` and then imported the agent, but static `import` statements are hoisted and run before any other code. `gemini.ts` read `process.env.GEMINI_CHAT_MODEL` into a constant at import time, before the env file loaded, so it silently fell back to the default model. I fixed it twice: the model name is now read lazily in a `getChatModel()` function, and the eval loads the agent with a dynamic `await import()` after loading env. The rule: never read `process.env` at the top level of a module a script may import early.

Testing, scale & trade-offs

Q46–Q50

Q46 How do you test an LLM app when the output changes every run?

I test behaviour, not exact strings. The eval suite runs 20 questions against the sample PDF, and each case checks things like: the answer contains one of the expected facts ("1,999"), it doesn't contain forbidden text, `searchDocs` was called, `createTicket` was not. Categories: grounded answers, paraphrases, questions the PDF doesn't cover, ticket consent, off-topic requests, two prompt-injection attacks and a Hindi question. I run it after any change to the prompt, chunk size or threshold.

Q47 What breaks first at 1,000 businesses, and how would you fix it?

1. **Synchronous upload.** A big PDF can exceed the 60-second function limit. Move extraction and embedding to a background job (a queue, or Supabase Edge Functions) and poll the document status.
2. **No rate limiting on the public chat.** Anyone with a link can burn the Gemini quota. Add per-IP and per-business limits with Upstash Redis.
3. **Paid LLM tier and cost control.** Free-tier quotas are far too small; move to a paid tier and cache answers to repeated questions.
4. **Vector search is fine.** HNSW with an `owner_id` index handles millions of rows, and the owner filter keeps each search small.

Q48 Pure vector search has weaknesses. How would you improve retrieval?

Vector search can rank exact terms like plan names, SKUs or prices poorly. I'd add hybrid search: Postgres full-text search alongside vector search, merged with reciprocal rank fusion. Beyond that, a re-ranking model on the top 20 results, and keeping tables intact as single chunks during ingestion.

Q49 What would you change with another month?

1. Replace the user id in the chat URL with a random business slug, so links don't expose internal ids.
2. Stream the final answer token by token to improve perceived speed.
3. Store conversations so owners can review them (today history lives only in the browser tab).
4. Add OCR so scanned PDFs work.
5. Let owners answer a ticket and add that answer to the knowledge base in one click.

Q50

Why Next.js, Supabase and Gemini? What would you swap?

Next.js gives the UI, API routes, server actions and middleware in one TypeScript codebase that deploys to Vercel with a git push. Supabase gives Postgres, pgvector, Auth, Storage and RLS in one service, so vectors live next to the data they describe instead of in a separate vector database. Gemini had a free tier with both chat and embedding models. Because the agent loop is hand-written and the model name is an env variable, swapping in another LLM is mostly one file, `gemini.ts`. A dedicated vector database like Pinecone only becomes worth it at a scale far beyond this.